Introduction

Decision trees partition the feature space by recursive binary splits, fitting a constant prediction in each terminal region (leaf). The Classification and Regression Trees (CART) algorithm by Breiman, Friedman, Olshen and Stone (1984) is the dominant implementation. Trees are interpretable (a single fitted tree can be read as a sequence of if-else rules), handle mixed numeric and categorical data without preprocessing, require no feature scaling, and serve as the building blocks for random forests and gradient boosting. Their main weakness is high variance — small data perturbations produce very different trees — which motivates ensemble methods that aggregate many trees.

Prerequisites

A working understanding of classification and regression problems, impurity measures (Gini, entropy, variance), and the bias-variance trade-off in model complexity.

Theory

At each node, CART searches over all features and candidate split points, choosing the split that maximally reduces impurity:

- **Regression**: minimise within-node variance.
- **Classification**: minimise Gini $1 - \sum_k p_k^2$ or entropy $-\sum_k p_k \log p_k$.

Trees grow recursively until leaves are pure or a minimum-size constraint is hit, then are pruned back using cost-complexity pruning. The cost-complexity criterion is

$$R_\alpha(T) = R(T) + \alpha |T|,$$

where $R(T)$ is the training error and $|T|$ the number of terminal leaves. The pruning parameter α is chosen by cross-validation; the standard rule selects the smallest tree within one CV-standard-error of the minimum.

Assumptions

Splits are axis-aligned (a single tree captures interactions only by combining splits at different depths, which limits its ability to model rotated or oblique decision boundaries); observations are independent.

R Implementation

```
library(rpart); library(rpart.plot)

set.seed(2026)
n <- 400
df <- data.frame(
  x1 = runif(n), x2 = runif(n),
  y  = factor(ifelse(runif(n) < plogis(-2 + 3 * runif(n) + 2 * runif(n)),
                     "pos", "neg"))
)

tree <- rpart(y ~ x1 + x2, data = df,
              method = "class",
              control = rpart.control(cp = 0.001))

printcp(tree)                  # CV error vs complexity
tree_pruned <- prune(tree,
```

```
                      cp = tree$cptable[which.min(tree$cptable[, "xerror"]), "CP"])
rpart.plot(tree_pruned, box.palette = c("#F4A261", "#2A9D8F"))
```

Output & Results

`rpart()` returns a fitted tree object; `printcp()` reports the cross-validation error at each candidate complexity parameter; `prune()` produces the cost-complexity-pruned tree at the chosen α. `rpart.plot()` visualises the tree with split conditions and leaf class proportions.

Interpretation

A reporting sentence: "CART with cost-complexity pruning at the CV-optimal α retained 5 leaves; the dominant split was on x_1 at 0.52, partitioning 78 % of positive cases into one branch and 80 % of negatives into the other; sub-splits on x_2 refined predictions within each branch." Always state the pruning method and the resulting tree size.

Practical Tips

- Always prune using the `xerror` column of the CP table; unpruned trees over-fit severely.
- A single tree has high variance; for predictive accuracy, prefer random forests, gradient boosting, or extra trees.
- Trees handle missing data via surrogate splits (rpart's default); this is one of the few methods that does so without explicit imputation.
- Visualise with `rpart.plot()` for `ggplot`-friendly output; `partykit::as.party()` provides a richer object with print and plot methods.
- For regression, use `method = "anova"`; for classification, `method = "class"`; for survival, `method = "exp"` and the `rpart` package has built-in support.
- For interpretability with better accuracy, use a small ensemble (e.g., 50 trees in a random forest) and aggregate variable-importance measures from across trees.

R Packages Used

`rpart` for CART implementation with cost-complexity pruning; `rpart.plot` for tree visualisation; `partykit` for an alternative tree framework with richer output classes; `tree` for an older alternative implementation; `caret` and `tidymodels::decision_tree()` for cross-validated pruning within ML pipelines.

For Reviewers

What to look for in a paper using this method.

- **Common misapplications.**
- **Diagnostics that should be reported but often aren't.**
- **Red flags in tables and figures.**
- **What to verify.**
- **What an adequate Methods paragraph must contain.**

See also — labs in this chapter

- Cross-validation, nested CV, bootstrap .632+
- Regularisation: ridge, lasso, elastic net
- Trees, random forests, gradient boosting
- Interpretability and SHAP
- Tabular neural networks with torch
- Imaging and sequence models intro
- tidymodels pipelines

- FDR, knockoffs, replication crisis